

Encapsulamento em POO: Teoria e Prática

Prof. Me. Hélio Esperidião

O que é Encapsulamento?

Definição:

Mecanismo que esconde os detalhes internos de uma classe/objeto, expondo apenas o necessário.

Analogia: Como uma "cápsula" que protege os dados.

O conceito faz parte dos **Pilares da POO**:

Abstração
Encapsulamento,
Herança,
Polimorfismo

Benefícios do Encapsulamento

Estados inválidos:

- Protege a classe de dados inválidos.

Manutenção:

- Facilita mudanças internas sem afetar outras classes.

Reusabilidade:

- Código mais organizado.

Segurança:

- Dados sensíveis protegidos.

Quando usar encapsulamento?

Basicamente **SEMPRE!**

A interface, ou seja, a forma como classes e objetos conversam um com o outro, deve sempre estar isolada da forma como executam o que se propuseram a fazer.

O que é a **quebra** de encapsulamento?

É quando a implementação de uma funcionalidade 'vaza' para diversas partes do sistema, com código em regiões, unidades, módulos ou pacotes("pastas") muito diferentes.

Sempre que precisamos modificar essa funcionalidade, necessitamos alterar diversas partes distantes do código.

Modificadores de acesso

Usado para **MODIFICAR** o modo como são declaradas as classes, métodos e variáveis.

Existem três modificadores de acesso e um padrão.

Toda classe, método e variáveis de instância que declara tem um controle de acesso.

Mediadores de acesso

Public – deixa visível a classe ou membro para todas as outras classes, subclasses e pacotes do projeto .

Private – deixa visível o atributo apenas para a classe em que este atributo se encontra.

Protected – deixa visível o atributo para todas as outras classes e subclasses que pertencem ao mesmo pacote.

Publico VS Private

Publico todas as classes podem usar os métodos e atributos

Privado apenas a classe pode usar os métodos e atributos

Será mais claro nas próximas aulas

Modificador	Visível na Própria Classe	Visível no Mesmo Pacote (pasta)	Visível em Subclasses (Herança)	Visível Globalmente (Qualquer Classe)
public	✓ Sim	✓ Sim	✓ Sim	✓ Sim
protected	✓ Sim	✓ Sim	✓ Sim	✗ Não
default (sem modificador)	✓ Sim	✓ Sim	✗ Não (só se estiver no mesmo pacote)	✗ Não
private	✓ Sim	✗ Não	✗ Não	✗ Não

Modificadores em Classes

Uma classe pode ser declarada apenas com acesso **public** ou **default**.

O acesso **public**, permite que todas as classes tenha acesso a classe, para acessar uma classe public que esteja em outro pacote precisamos usar o import.

Quais os modificadores dos atributos, métodos e da classe?

// Declaração da classe "Quadrado". Ela representa o conceito de um quadrado.

```
class Quadrado {
```

// Atributo da classe: armazena o tamanho do lado do quadrado.

// É do tipo float para permitir valores decimais.

```
float lado;
```

```
float calcularArea() {  
    return lado * lado;  
}
```

```
float calcularPerimetro() {  
    return lado * 4;  
}
```

```
float calcularDiagonal() {  
    float raizDois = (float) Math.sqrt(lado * lado + lado * lado);  
    return (float) raizDois;  
}
```

```
}
```

Quadrado
lado: float
calcularArea(): float calcularDiagonal(): float calcularPerimetro(): float

C Quadrado
△ lado : float
▲ calcularArea() : float ▲ calcularPerimetro() : float ▲ calcularDiagonal() : float

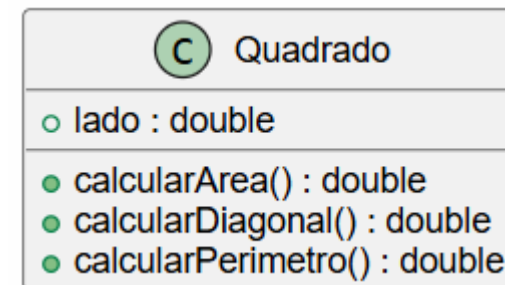
Atributos e métodos públicos

Classe Quadrado

```
class Quadrado {  
    public double lado;  
    public double calcularArea(){  
        return lado*lado;  
    }  
    public double calcularDiagonal(){  
        return Math.sqrt(lado*lado + lado *lado);  
    }  
    public double calcularPerimetro(){  
        return 4*lado;  
    }  
}
```

Classe de teste

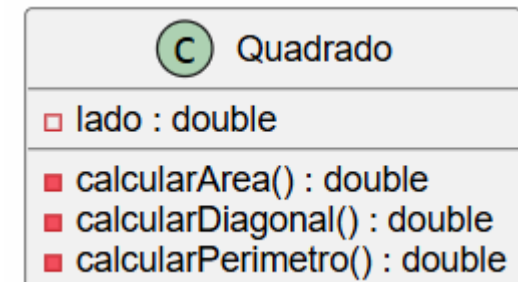
```
public class Teste {  
    public static void main(String[] args) {  
        Quadrado q = new Quadrado();  
        q.lado = 5.25;  
        System.out.printf("%.4f",q.calcularArea());  
    }  
}
```



Métodos e Atributos privados não podem ser acessados de fora da classe

```
class Quadrado {  
    private double lado;  
    private double calcularArea(){  
        return lado*lado;  
    }  
    private double calcularDiagonal(){  
        return Math.sqrt(lado*lado + lado *lado);  
    }  
    private double calcularPerimetro(){  
        return 4*lado;  
    }  
}
```

```
public class Teste {  
    public static void main(String[] args) {  
        Quadrado q = new Quadrado();  
        q.lado = 5.25;  
        System.out.printf("%.4f",q.calcularArea());  
    }  
}
```



Todos os atributos e métodos da classe **Quadrado** são privados.

Todos os atributos e métodos da classe **NÃO** podem ser utilizados fora da classe quadrado.

Encapsulando dados

Para acessar atributos privados é necessário que existam dois métodos associados a cada atributos.

Método **get** Recuperar o valor de um atributo.

Método **set** atribui o valor de um atributo.

this

Faz referência a própria instância, ou seja o this é o próprio objeto.

É utilizado para referenciar atributos e métodos da instância.

Método SET

Os atributos são privados

O nome do método é iniciado com set seguido do nome do atributo com a primeira letra maiúscula

O método set é público

```
public class Pessoa {  
    private int idade;  
    private String nome;  
    private Pessoa mae;  
    public void setIdade(int idade) {  
        this.idade = idade;  
    }  
}
```

this faz referencia ao atributo privado idade

O método é void pois não retorna nenhum valor, apenas modifica o valor do atributo idade

Método GET

Como o atributo idade é um inteiro o método vai retornar um inteiro

```
private int idade;  
private String nome;  
private Pessoa mae;  
public void setIdade(int idade) {  
    this.idade = idade;  
}  
public int getIdade() {  
    return this.idade;  
}
```

this -> "do objeto" , "deste objeto"

"doObjeto".idade

"desteObjeto".idade

O nome do método é iniciado com get seguido do nome do atributo com a primeira letra maiúscula

Retorna o valor do atributo

Completo

```
package poo;

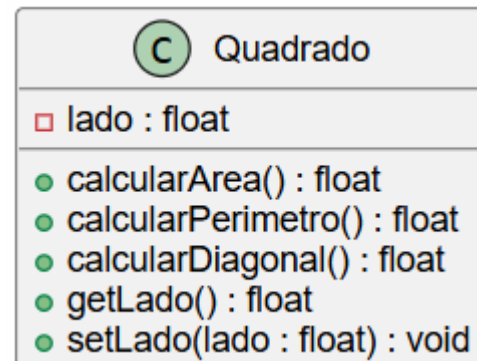
public class NomeClasse {
    public static void main(String[] args){
        Pessoa p =new Pessoa();
        p.setIdade(18);
        System.out.println(p.getIdade());
    }
}
```

```
public class Pessoa {
    private int idade;
    private String nome;
    private Pessoa mae;
    public void setIdade(int idade) {
        this.idade = idade;
    }
    public int getIdade() {
        return this.idade;
    }
    public String getNome() {
        return this.nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public Pessoa getMae() {
        return this.mae;
    }
    public void setMae(Pessoa mae) {
        this.mae = mae;
    }
    public void fazerAniversario(){
        this.idade++;
    }
}
```

Encapsular os atributos da classe Quadrado

```
//declaração de classe publica.
public class Quadrado {
    private float lado;
    public float calcularArea() {
        return this.lado * this.lado;
    }
    public float calcularPerimetro() {
        return this.lado * 4;
    }
    public float calcularDiagonal() {
        float raizDois = (float) Math.sqrt(this.lado * this.lado + this.lado * this.lado);
        return (float) raizDois;
    }
    public float getLado() {
        return this.lado;
    }

    public void setLado(float lado) {
        this.lado = lado;
    }
}
```



- lado é um atributo privado.
- lado só pode ser acessado de dentro de uma instância de quadrado.
- Para modificar ou acessar o valor do atributo lado é necessário utilizar os métodos de set/get
- O método setLado(double lado) pode modificar o valor do atributo privado lado.
- O método getLado() pode recuperar o valor do atributo privado lado.
- Observe que o método setLado(double lado) recebe a variável lado, this lado faz referencia ao atributo privado lado.

Usar a classe quadrado

```
class Teste {  
    public static void main(String[] args) {  
        Quadrado q1 = new Quadrado();  
        q1.setLado(2.35);  
        System.out.printf("Área = %.4f",q1.calcularArea());  
    }  
}
```

Validação de dados

- O uso de métodos de acesso get/set podem ser utilizados para proteger a aplicação e impedir que dados inconsistentes sejam utilizados pelo objeto.

```
public class Quadrado {
    private float lado;

    public float calcularArea() {
        return this.lado * this.lado;
    }

    public float calcularPerimetro() {
        return this.lado * 4;
    }

    public float calcularDiagonal() {
        float raizDois = (float) Math.sqrt(this.lado * this.lado + this.lado * this.lado);
        return (float) raizDois;
    }

    public float getLado() {
        return this.lado;
    }

    public void setLado(float lado) {
        if (lado <= 0) {
            throw new IllegalArgumentException("O lado do quadrado deve ser um valor positivo");
        }
        this.lado = lado;
    }
}
```

C	Quadrado
□	lado : float
●	calcularArea() : float
●	calcularPerimetro() : float
●	calcularDiagonal() : float
●	getLado() : float
●	setLado(lado : float) : void

```
public class QuadradoMain {  
    public static void main(String[] args) {  
        Quadrado q = new Quadrado();  
  
        System.out.println("Tentando setLado(-5.0)...");  
        q.setLado(-5.0f); // Lançará IllegalArgumentException e encerrará o programa  
        System.out.println("Esta linha não será executada.");  
    }  
}
```

Observe que a classe possui um mecanismo de proteção, isso impede o uso indevido da classe.
Um valor de dado incorreto é enviado e a classe não permite que a execução continue.

```
⊗ Tentando setLado(-5.0)...  
Exception in thread "main" java.lang.IllegalArgumentException: O lado do quadrado deve ser um valor positivo  
    at Quadrado.setLado(Quadrado.java:23)  
    at QuadradoMain.main(QuadradoMain.java:8)  
❖ PS D:\professor\2025\FEAU\Programação II\ProgramacaoII2025\Aula03> []
```

O programador é obrigado a criar uma estrutura que força a validação dos dados para impedir que o uso incorreto da classe encerre a execução do problema.

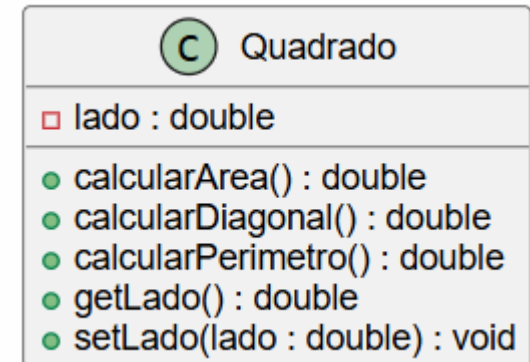
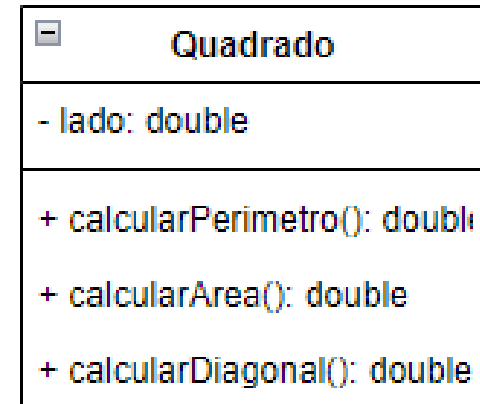
```
public class QuadradoMain {  
  
    public static void main(String[] args) {  
        Quadrado q = new Quadrado();  
        float novoLado = -5.0f;  
        if (novoLado > 0) {  
            q.setLado(novoLado);  
        } else {  
            System.out.println("Lado inválido");  
        }  
    }  
}
```

- ta\Roaming\Code\User\workspaceStorage\a8bda0f1b5fdd330531ea1faba7a66d3\redhat.java\jdt_ws\Aula03_2d96c0a4\bin' 'QuadradoMain2'
- Lado inválido
- PS D:\professor\2025\FEAU\Programação II\ProgramacaoII2025\Aula03>

UML

```
class Quadrado {  
    private double lado;  
    public double calcularArea(){  
        return lado*lado;  
    }  
    public double calcularDiagonal(){  
        return Math.sqrt(lado*lado + lado *lado);  
    }  
    public double calcularPerimetro(){  
        return 4*lado;  
    }  
    public double getLado() {  
        return lado;  
    }  
    public void setLado(double lado) {  
        this.lado = lado;  
    }  
}
```

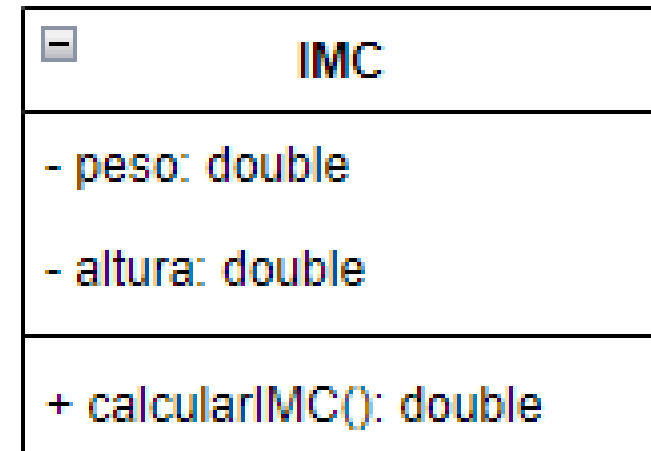
- Significa **privado**
- + Significa **público**
- # Significa **protegido**



UML

```
class Imc {  
    private double altura;  
    private double peso;  
    public double CalcularImc(){  
        return peso/(altura*altura);  
    }  
    public double getAltura() {  
        return altura;  
    }  
    public void setAltura(double altura) {  
        this.altura = altura;  
    }  
    public double getPeso() {  
        return peso;  
    }  
    public void setPeso(double peso) {  
        this.peso = peso;  
    }  
}
```

- Significa privado
+ Significa público

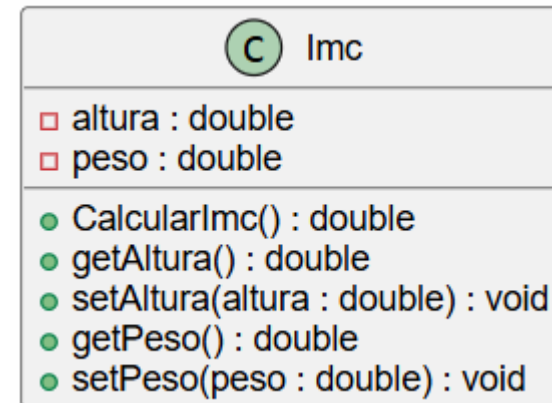


puml

```
@startuml
class Imc {
- altura : double
- peso : double

+ CalcularImc() : double
+ getAltura() : double
+ setAltura(altura : double) : void
+ getPeso() : double
+ setPeso(peso : double) : void
}
@enduml
```

- Significa **privado**
- + Significa **público**
- # Significa **protegido**



Encapsulamento não é apenas get e set

- Get e Set é fundamental e é uma recomendação para acesso a atributos de um objeto.
- Encapsulamento é mais do que isso!
- Encapsulamento é também **esconder detalhes da classe** do usuário(programador)

```

public class Bhaskara {
    private double a;
    private double b;
    private double c;
    private double calcularDelta() {    // Método privado para calcular delta
        double delta = Math.pow(b, 2) - 4 * a * c;
        if (delta < 0) {
            throw new ArithmeticException("Não existem raízes reais (delta < 0)");
        }
        return delta;
    }
    public double[] calcularRaizes() {
        double delta = calcularDelta();
        return new double[] {
            (-b + Math.sqrt(delta)) / (2 * a),
            (-b - Math.sqrt(delta)) / (2 * a)
        };
    }
    public double getA() {    // Métodos get e set com validação
        return a;
    }
    public void setA(double a) {
        if (a == 0) {
            throw new IllegalArgumentException("0 coeficiente 'a' não pode ser zero");
        }
        this.a = a;
    }
}

```

```

public static void main(String[] args) {
    Bhaskara eq = new Bhaskara();

    // Teste com raízes reais distintas
    eq.setA(1);
    eq.setB(-5);
    eq.setC(6);

    System.out.println(eq.calcularRaizes()[0]);
    System.out.println(eq.calcularRaizes()[1]);
}

```

Bhaskara

- a : double
- b : double
- c : double

- calcularDelta() : double
- calcularRaizes() : double[]
- getA() : double
- setA(a : double) : void

O que é static em Java?

Quando definimos atributos com a palavra *static* em uma classe ela terá um comportamento especial: **ela será a mesma para todos os objetos daquela classe.**

Não haverá um valor para ela em cada objeto.

Todos os objetos, ao acessarem e modificarem essa variável, acessarão a mesma variável, o mesmo espaço da memória, e a mudança poderá ser vista em todos os objetos.

Variáveis e métodos estáticos

Pertencem à classe e não a uma instância.

Isso quer dizer que eles podem ser utilizados sem ter que instanciar a classe.

As variáveis estáticas são compartilhadas por todos os objetos.

Parece estranho mas, Você já utiliza isso.

Observe o método `sqrt` da classe `Math`.

- `Sqrt` é um método `estático`, não é necessário instanciar `Math` para o seu acesso.
- O mesmo acontece com `PI`, que é um atributo estático da classe `PI`.

```
Math.sqrt(25);
```

```
double x = Math.PI
```


Variáveis estáticas - Contadores



Algumas variáveis estáticas devem mudar à medida que o programa é executado.



Os contadores são comuns exemplos disso.



Queremos que a contagem aumente com o tempo

Variáveis estáticas - Constantes

Algumas variáveis estáticas nunca devem mudar durante o programa.

Esse tipo de variável capaz é conhecido como uma constante .

Ele usa o modificador `final` para garantir que a variável nunca mude.

Constantes *final* estáticas usam uma convenção de nomenclatura diferente de outras variáveis.

Usam todas as letras maiúsculas com sublinhado entre "palavras".

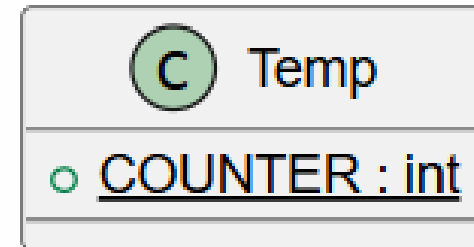
Quando usar variáveis static em Java

- Quando você quiser ter um controle sobre os objetos ou quando todos os objetos devem partilhar uma informação.

Exemplo

```
public class Temp {  
    public static int COUNTER = 0;  
}
```

```
@startuml  
class Temp {  
    + {static} COUNTER : int  
}  
@enduml
```



Exemplo

```
class Led {  
    public static final int HIGH= 1;  
    public static void main(String[] args) {  
        System.out.println(Led.HIGH);  
    }  
}
```

Exemplo de métodos estáticos

```
public class ExemploEstatico {  
    public static int CONTADOR=1;  
    public static int somarInteiros(int x, int y){  
        return x+y;  
    }  
}
```

Atributo estático

Método estático

```
package javaapplication1;  
  
public class JavaApplication1 {  
    public static void main(String[] args) {  
        // TODO code application logic here  
        int z = ExemploEstatico.somarInteiros(50, 10);  
        System.out.println(z);  
        ExemploEstatico.CONTADOR++;  
        System.out.println(ExemploEstatico.CONTADOR);  
    }  
}
```

Não é necessário instanciar
para usar métodos e atributos
estáticos